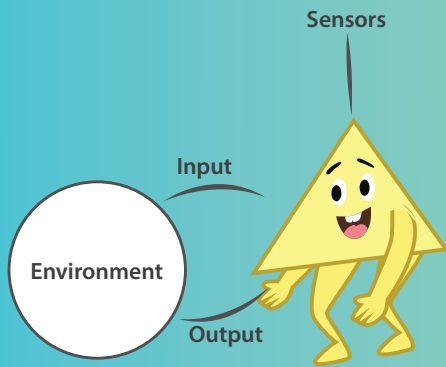


Applications of Vectors

06

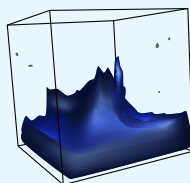
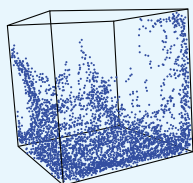


Ever wondered how we bring the real world into games, VR, or even training simulations? With the increasing demand for virtual worlds, recreating the real world on digital platforms is a crucial skill. Let us explore the world of vectors and how they help bring imagination to life on digital screens!



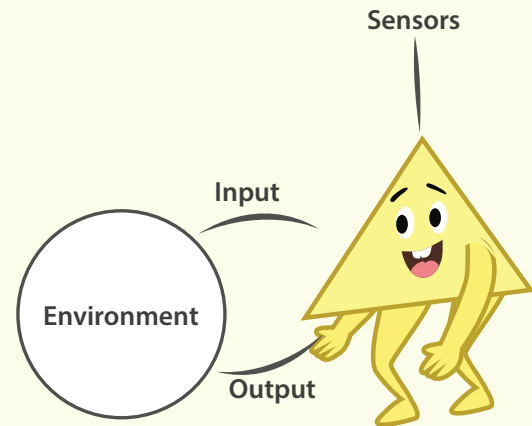
One key aspect of applying vector concepts in digital simulations is the use of real-world formulas, like the laws of physics, to create realistic effects.

For example, computer scientists use algorithms to simulate complex phenomena like fire, smoke, and light rays. They start with simple particles and apply basic physical laws, allowing them to function autonomously. By multiplying these entities, we can simulate realistic effects.

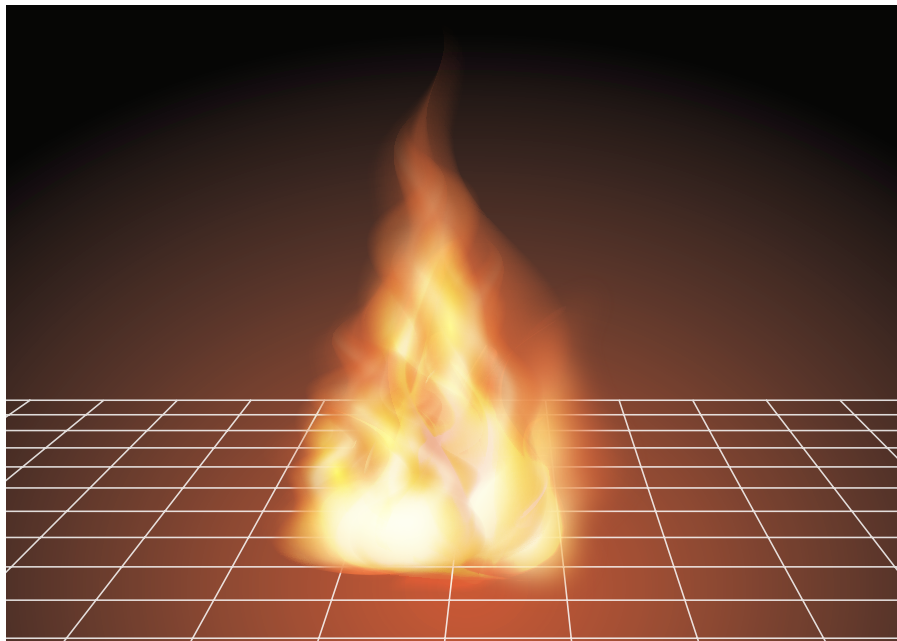


Recall:

We have learnt that autonomous agents are entities capable of generating their own movements and actions, simulating real-world behaviours in virtual environments. Unlike pre-defined animations, these agents use algorithms and rules to act dynamically in real time.



When grouped together, these agents form what are known as **particle systems**, used to simulate effects like fire, smoke, and water. To create autonomous agents in p5.js, we will implement concepts such as Object-Oriented Programming and Vectors.



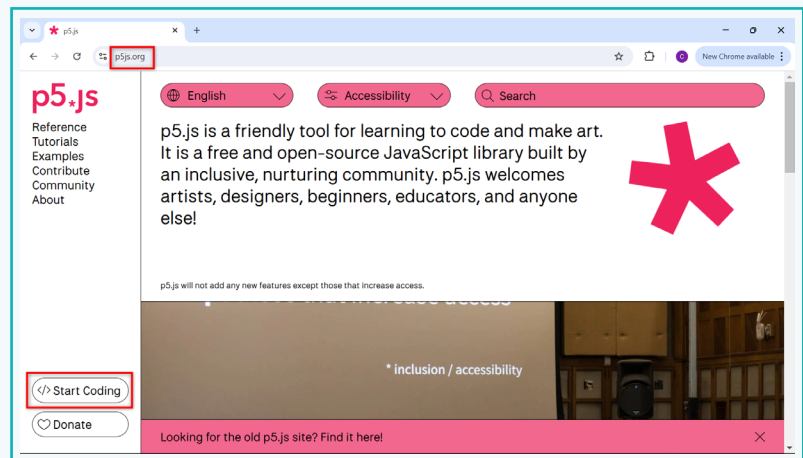
In this lesson, we will create a particle agent that follows the basic physical laws of motion and gravity. With just one particle, we can create a source point that generates particles continuously. This will serve as the building block for future simulation projects.

Activity

Part 1 - Open Code Editor

We will start by creating a class for a moving agent. This class can exhibit all the necessary properties and methods for movement and rendering.

- 1 Open your web browser and go to the p5.js website: <https://p5js.org/>.
- 2 Click on the 'Start Coding' button to open the p5.js Web Editor.
- 3 In the editor, click on the current project name in the top-left corner, rename it to 'vectorApp' and press 'Enter'.



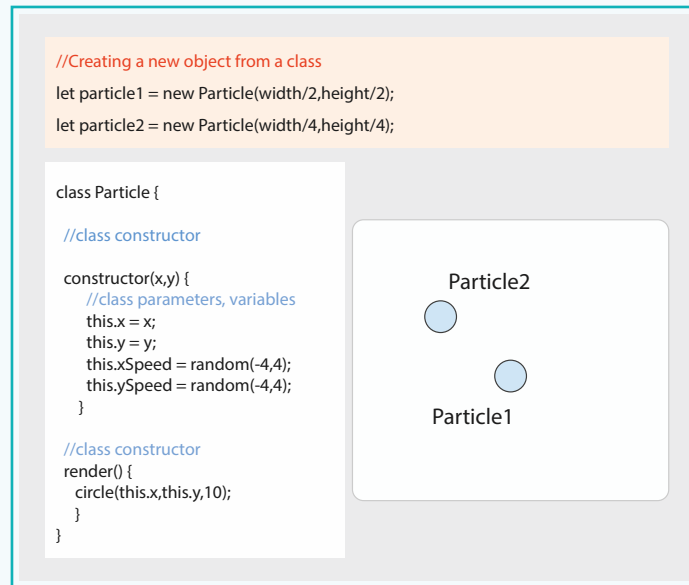
Recall:

To implement the OOP concepts and create an agent, follow this general approach:

1. **Define the Agent Class:** This class represents the autonomous agent containing properties of an agent such as position, velocity, acceleration, and other relevant attributes.
2. **Add Class Methods:** Include methods to display the agent and update its state to perform actions.
3. **Create Object:** Generate objects from the agent class and call the class methods to render them.
4. **Define Behaviors:** Implement methods within the class to update the agent's position, velocity, and acceleration, allowing it to behave autonomously.

Activity

Here is a sample code demonstrating how to create a class and its objects.

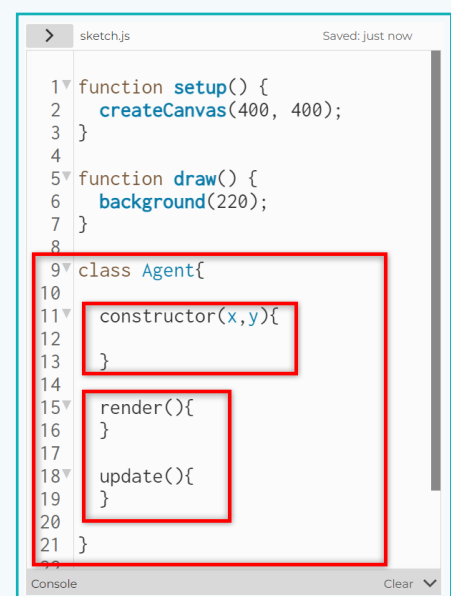


All the algorithms implement and combine forces to influence the agent's acceleration or velocity.

Part 2 - Create an Agent Class

Let us create the class for the particle agent with its base properties and render method.

- 1 Create a class *Agent* and add a default constructor method.
- 2 Add the input constructor arguments (x,y) to set the initial position of the class object.
- 3 Add methods *render()* and *update()* in the class.



With this basic class definition, motion dynamics can be incorporated through the implementation of laws of physics.

06. Applications of Vectors

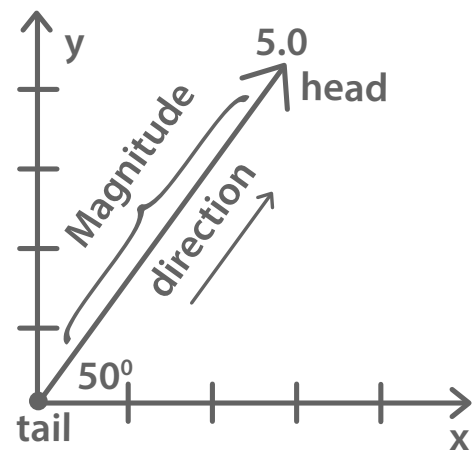
We have studied the concepts of speed, velocity, acceleration, and other related concepts that are fundamental to the physical laws of motion. Let us revisit these concepts and their implementation in code.

Scalar vs Vector

Physical quantities like speed, which have only magnitude, are called **Scalars**. In contrast, physical quantities that have both magnitude and direction, such as velocity, are called **Vectors**.

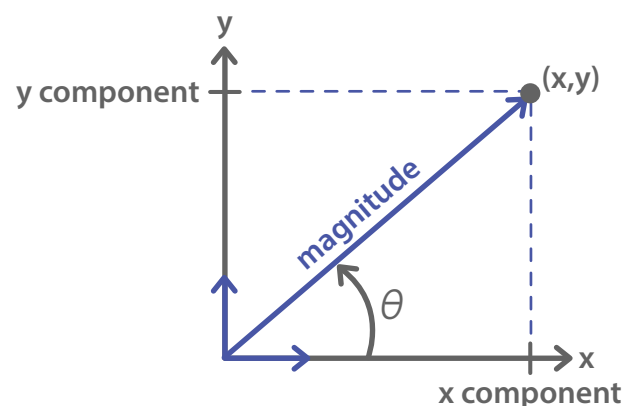
Vectors

Vectors are essential for representing physical quantities like position, velocity, force, and acceleration within a specific coordinate system. A vector is depicted as a line segment with an arrow. The direction of the arrow indicates the vector's direction, while the length of the segment represents its magnitude.



Components

A vector can be divided into its components along various axes of a coordinate system. In 2-dimensional space, it can be expressed through its x and y components. For instance, the x and y coordinates of an object correspond to the x and y components of its position vector.



Relation between vectors and components

A vector in a 2-dimensional space can always be represented using its x and y components. Properties that are vectors—i.e., those that have direction, such as position, velocity, or acceleration — are represented using the x and y components.

For example,

Position	xPos and yPos
Velocity	xVel and yVel
Acceleration	xAcc and yAcc

Creating a Vector

In p5js, we can create a vector by calling the `createVector()` function and passing the initial x, y (and optionally, z) coordinates as arguments. This approach eliminates the need to create separate variables for the x and y components for each physical entity.

```
var pos = createVector(0,0);
```

Individual components of the vectors can be accessed by appending the `.x` and `.y` operators to the vector name, such as `pos.x` and `pos.y`.

Activity

Part 3 - Add the Class Properties

Each property of the class that we define in the constructor, or we use in methods must prepend *this*. keyword.

- 1 Create a property variable `this.pos` in the constructor for the position vector using the `createVector(x,y)` method with x and y as input arguments.
- 2 Create another property `this.r` to set the base size of the shape and initialise it to '10'.

Activity

- 3 Use the ellipse method to draw a circle at positions `this.pos.x` and `this.pos.y`.



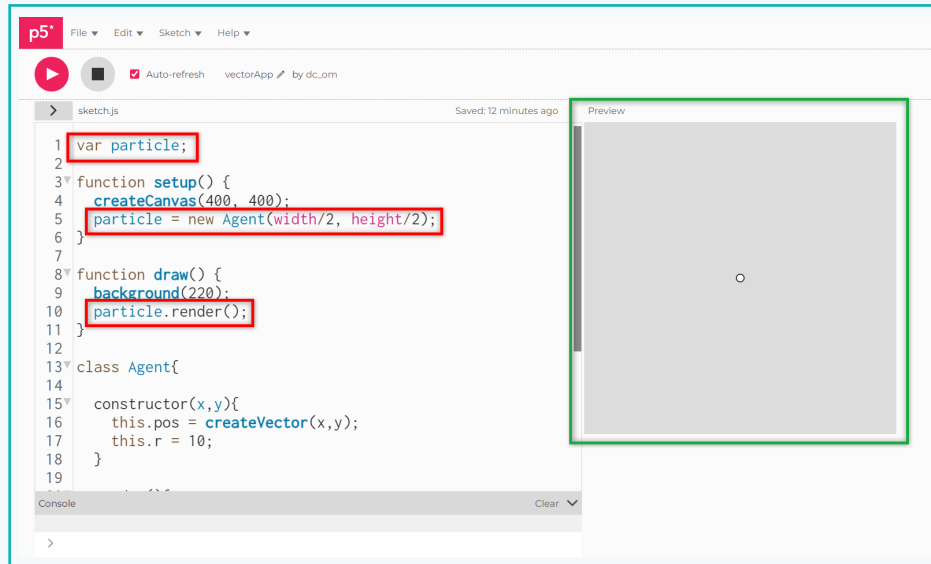
```
> sketch.js Saved: just now
7
8 function draw() {
9   background(220);
10 }
11
12 class Agent{
13
14   constructor(x,y){
15     this.pos = createVector(x,y);
16     this.r = 10;
17   }
18
19   render(){
20     ellipse(this.pos.x, this.pos.y, this.r);
21   }
22
23   update(){
24   }
25
26 }
```

Part 4 - Render the Object

To be able to see the output, we must create an object for the class *Agent* and call the *render()* method.

- 1 Define a variable *particle* at the beginning to create and save an object of the *Agent* class.
- 2 In the *setup()* function, create a new object of class *Agent* with input as *width/2* and *height/2* as the initial position of the *particle* object.
- 3 Call the *render()* method in the *draw()* function using the dot operator: *particle.render()*.

Activity



This will display a circle at the centre of the canvas, representing the particle object. The use of vectors greatly simplifies the code.

Part 5 – Update the Position with Velocity Vector

Vector Operations: To move an object, we apply velocity, which describes how it changes position over time. In coding, each cycle of code execution, i.e. time for rendering an output frame, is considered as the unit time. Hence, the **velocity vector** represents the object's displacement from its original position in each frame. In simple language, we can state it as:

$$\text{New Position} = \text{Old position} + \text{Change in Position}$$

Implementing this in our code, we can get a new position vector as:

$$\text{New Position Vector} = \text{Current Position Vector} + \text{Velocity Vector}$$

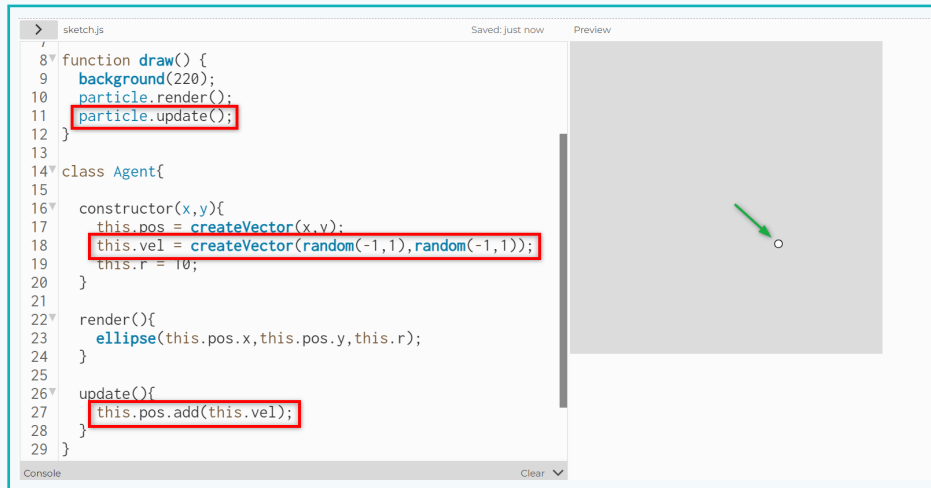
Instead of using regular operators like '+', we use vector methods like `add()` in p5.js for such operations. Therefore, we can update the position of the object using vector addition as:

this.pos.add(this.vel);

- 1 Create a new vector property `this.vel` in the constructor method with the `x` and `y` components as random values between `-1` and `1`. This applies the velocity to the particle in a random direction.
- 2 Update the `this.pos` vector by adding the `this.vel` vector using the `add()` method in the `update()` method.

Activity

- 3 Call the *update()* method for the *particle* object in the *draw()* function.



```
1 > sketch.js Saved: just now Preview
2 /
3
4 8 function draw() {
5   background(220);
6   particle.render();
7   particle.update();
8 }
9
10
11
12
13
14 class Agent{
15
16   constructor(x,y){
17     this.pos = createVector(x,y);
18     this.vel = createVector(random(-1,1),random(-1,1));
19     this.r = 10;
20   }
21
22   render(){
23     ellipse(this.pos.x,this.pos.y,this.r);
24   }
25
26   update(){
27     this.pos.add(this.vel);
28   }
29 }
```

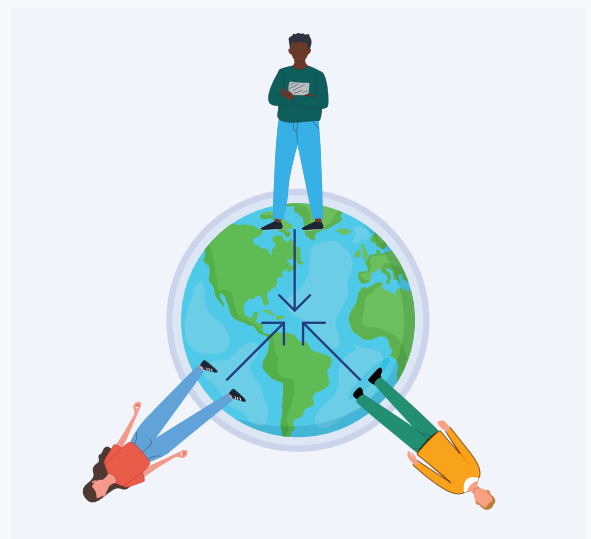
Note:

Particle will move in random directions at varying velocities. Now, we can apply the force of gravity to the particle.

Part 6 - Add Gravity

Gravity

Gravity is the force pulling objects towards the centre of the Earth or any celestial body, causing acceleration. The acceleration caused due to gravitational force, commonly denoted by 'g', affects all objects equally on Earth, regardless of mass. Hence, all objects within the effect of gravity around the Earth fall at the same rate.



Activity

Acceleration is the change in velocity per unit time. Velocity is updated by adding acceleration, similar to how location is changed by adding velocity.

$$\text{New Velocity Vector} = \text{Current Velocity Vector} + \text{Acceleration Vector}$$

Let us add an acceleration vector to the code to add the movement due to gravity.

- 1 Create a *this.acc* property vector in the class constructor using the *createVector()* command with its initial x and y components set to '0' and '0.05'. The y component value of '0.05' will generate the force downwards to simulate gravity.
- 2 Add *this.vel.add(this.acc)* before updating the position, so that the velocity gets updated by the acceleration.



```
13 class Agent{
14   constructor(x,y){
15     this.pos = createVector(x,y);
16     this.vel = createVector(random(-1,1),random(-1,1));
17     this.acc = createVector(0,0.05);
18     this.r = 10;
19   }
20   render(){
21     ellipse(this.pos.x,this.pos.y,this.r);
22   }
23   update(){
24     this.vel.add(this.acc);
25     this.pos.add(this.vel);
26   }
27 }
28
29
30
31
32
```

Even if the particle begins to move in multiple directions, it will fall.

Part 7 – Add Particle Lifespan

Particles need to be removed after falling out of sight. Assign a lifespan to each particle, after which it will not be visible. Let us implement this.

- 1 Create a *this.life* property in the class constructor and set its value to '255' indicating that after 255 frames, the particle's life should end.

Activity

- 2 Add a method `isLifeOver()` to the `Agent` class and add the `if – else` condition inside.
- 3 Check if `'this.life < 0'`. If it is, return `'true'`; otherwise, return `'false'`. This way, the function will return the Boolean values as `'true'` or `'false'` based on the condition inside
- 4 In the `update` method, use the `'decrement'` operator to reduce the value of the `this.life` property variable by `'2'`.

```

> sketch.js Saved: just now
19 this.acc = createVector(0,0.05);
20 this.r = 10;
21 this.life = 255;
22 }
23
24 render(){
25   ellipse(this.pos.x,this.pos.y,this.r);
26 }
27
28 isLifeOver(){
29   if(this.life < 0){
30     return true;
31   }else{
32     return false;
33   }
34 }
35
36 update(){
37   this.life -= 2;
38   this.vel.add(this.acc);
39   this.pos.add(this.vel);
40 }
41 }
Console Clear
```

Let us understand how to remove or delete an object when a particle's life is over.

Activity

Part 8 – Create Array of Objects

JavaScript does not allow explicit deletion of objects created from a class. Instead, remove all references to an object to allow the JavaScript garbage collector to reclaim the memory.

Bytes

A **reference to an object** means the link or connection used to interact with that object in the code. When an object is created with a name like 'particle' in our code, it is the reference to the object to use it in our code. For example,

let obj = new MyClass();

Here, *obj* is the reference, or the name used to interact with that object.

The **JavaScript garbage collector** automatically frees memory by removing objects and variables that are no longer needed. It clears objects and variables without references, preventing memory leaks and keeping programs efficient without manual memory management.

So instead of deleting a single object, we will first create an array of particle objects.

- 1 Change the variable declaration of *particle* to *particles* as an array.
- 2 In setup, comment out the code to add new object. Instead, create a new object from the *Agent* class in the *draw* function. Add it to the *particles* array using the *push()* command.
- 3 Include a call for the *render* and *update* methods in a *for* loop, so that the methods are called for all elements of the *particles* array.



Particles will continuously spawn and fall downwards due to gravity. Adjusting the velocity vector values can show different paths that particles follow under the effect of gravity.

Activity

Part 9 – Update Array of Objects

Add a condition to check if particle life is over and remove it. To remove an element or object from an array, we use the `splice()` method.

`splice(i, n)`: This JavaScript method modifies an array by changing or removing elements. Here, 'i' is the index from which to start changing or removing elements, and 'n' is the number of elements to remove starting from index 'i'. This method alters the original array and returns the updated array by removing mentioned elements.

- 1 Add an *if* statement within the *for* loop using the *isLifeOver()* method as its condition for each element in the *particles* array.
- 2 If the condition is satisfied, we can remove the *i*th object element from the array using the *splice()* method as *particles.splice(i, 1)*.



The screenshot shows a p5.js sketch editor with a code editor on the left and a preview window on the right. The code editor contains the following JavaScript code:

```
8 function draw() {  
9   background(220);  
10  particles.push(new Agent(width/2, height/2));  
11  
12  for(let i=0;i<particles.length;i++){  
13    particles[i].render();  
14    particles[i].update();  
15  
16    if(particles[i].isLifeOver()){  
17      particles.splice(i,1);  
18    }  
19  }  
20 }  
21  
22 class Agent{  
23  
24  constructor(x,y){  
25    this.pos = createVector(x,y);  
26    this.vel = createVector(random(-1,1),random(-1,1));  
27    this.acc = createVector(0,0.05);  
28  }  
29 }  
30
```

The code is saved 25 seconds ago. The preview window shows a gray background with a cluster of small white circles representing particles. A red box highlights the `if(particles[i].isLifeOver()){ particles.splice(i,1); }` code block in the code editor.

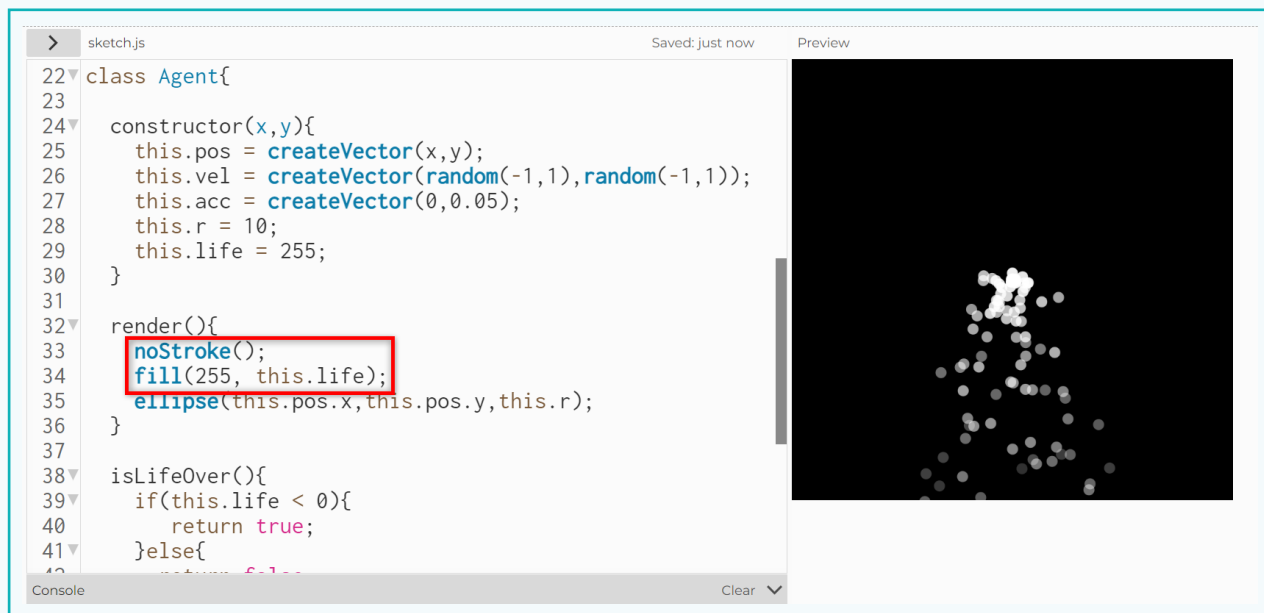
This way, the number of particles will not increase indefinitely. Also, once the object element is removed from the array, it loses all references and is deleted by the garbage collector.

Activity

Part 10 – Update Visuals

We can add some visual effects to this particle system to make it look interesting.

- 1 Change the canvas colour by updating value in `background()` command to 0.
- 2 Also, add the `noStroke()` and `fill(255,this.life)` commands to remove the border and to fill color, respectively. Reducing the alpha value in the variable `this.life` in the `fill` command will make particle colour transparent as its life diminishes.



With this, we have successfully created a particle system using vectors and object-oriented programming (OOP) in p5.js. We defined a particle class, applied vector operations for movement, and implemented gravity and lifespan, all while managing memory efficiently with JavaScript's garbage collection. We will use this basic setup to create interesting simulations and animations in further lessons.

Exercise

State if the statements are True or False.

- 1 To create a 2D vector using x and y component values, createVector(x,y) command is used.



- 2 If a property vector named this.pos is created in a class, its x and y components can be accessed inside the class as this.x and this.y.



Tick mark the correct answer.

- 3 _____ command is used to create a vector.

a. createVector() b. mult() c. limit() d. add()

- 4 A vector has magnitude as well as _____.

a. force b. direction c. scale d. none of the above

- 5 To add two vectors, we can use _____.

a. + operator b. mult() c. add() d. Both a and c

Answer the following questions in one line.

- 6 What is garbage collector in JavaScript?

- 7 How to use the splice() method?

Tech Flash

Autonomous Agents	: These are virtual objects with autonomy in generating their own movements and actions.
Scalars	: These are the physical entities that only have magnitude.
Vectors	: These are physical entities that have magnitude as well as direction.
Components of a vector	: These are the projections of the vector along different axes of a coordinate system.
Velocity	: It is the rate of change of displacement per unit time.
Acceleration	: It is the rate of change of velocity per unit time.
Garbage collector	: It automatically looks for objects and variables that are no longer needed in the code and clears them from memory, making room for new data.
Splice method	: It modifies an array by changing or removing mentioned elements.